

Lab Task 1: Hospital Patient Priority Queue (AVL Tree)

Hospitals need to manage patient admissions dynamically while ensuring that **critical patients (lower priority numbers)** are treated first.

Since new patients are constantly being added and discharged, the system must always stay efficient and balanced — ensuring fast insertion, lookup, and removal regardless of input order.

To achieve this, the hospital uses an **AVL Tree** as its internal data structure, where:

- Each node represents a patient.
- The tree is **ordered by patient priority** (lower number = higher priority).
- The tree automatically **rebalances itself** after every insertion and discharge to maintain $O(\log n)$ performance.

Data Model

Each AVL node represents one patient record.

```
struct Patient {  
    int priority;           // Smaller = more urgent  
    char name[50];         // Patient name  
    char condition[100];   // Description (e.g., "Fracture", "Heart  
Attack")  
};
```

Each AVL node will store:

```
struct Node {  
    Patient data;  
    Node* left;  
    Node* right;  
    int height;  
};
```

Note: Each node *wraps* a **Patient** and adds metadata needed for AVL balancing.

Functional Requirements

1. Insertion — `addPatient()`

- Inserts a new patient into the AVL Tree using `priority` as the key.
- Smaller numbers represent higher urgency.
- Reject duplicate priority numbers and return `false`.
- Automatically performs rebalancing if imbalance occurs.

2. Deletion — `removePatient()`

- Removes a patient with the given priority.
- Rebalances the tree after removal.
- If no such patient exists, return `false`.

3. Search — `findPatient()`

- Returns a pointer to the `Patient` struct if found.
- Returns `nullptr` if not found.

4. Highest and Lowest Priority Queries

- `getHighestPriorityPatient()` → returns the patient with the **lowest numeric priority value**.
- `getLowestPriorityPatient()` → returns the patient with the **highest numeric priority value**.
- If the tree is empty, return a `Patient` with `priority = -1`.

5. Range Query — `getPatientsInRange(low, high, outCount)`

- Returns all patients whose priority value lies between `low` and `high` (inclusive).
- Results are stored in a **dynamically allocated array** (`new Patient[count]`), ordered by increasing priority.
- Caller must free memory using `delete[]`.

6. Diagnostics

- `isBalanced()` — returns `true` if all balance factors $\in \{-1, 0, 1\}$.
- `getTotalPatients()` — total number of nodes.
- `getHeight()` — current tree height.

7. Display

- `displayInorder()` — prints all patients (priority, name, condition) in ascending order of priority.

Test Cases

```
#include "gtest/gtest.h"

#include "PatientAVL.h"

// 1. Add and Find Patient

TEST(PatientAVLTest, AddAndFindPatient) {

    PatientAVL avl;

    EXPECT_TRUE(avl.addPatient(5, "Alice", "Fracture"));

    EXPECT_TRUE(avl.addPatient(2, "Bob", "Heart Attack"));

    Patient* p = avl.findPatient(5);
```

```

    ASSERT_TRUE(p != nullptr);

    EXPECT_STREQ(p->name, "Alice");

    EXPECT_STREQ(p->condition, "Fracture");

    EXPECT_EQ(p->priority, 5);
}

// 2. Reject Duplicate Priorities
TEST(PatientAVLTest, RejectDuplicatePriorities) {
    PatientAVL avl;

    EXPECT_TRUE(avl.addPatient(10, "X", "Headache"));

    EXPECT_FALSE(avl.addPatient(10, "Y", "Fever"));

    EXPECT_EQ(avl.getTotalPatients(), 1);
}

// 3. Auto-Balancing After Multiple Inserts (LR Rotation)
TEST(PatientAVLTest, AutoBalancingAfterInsert) {
    PatientAVL avl;

    avl.addPatient(30, "A", "Cold");

    avl.addPatient(10, "B", "Flu");

    avl.addPatient(20, "C", "Asthma"); // triggers LR rotation

    EXPECT_TRUE(avl.isBalanced());
}

```

```
// 4. Delete Patient and Maintain Balance

TEST(PatientAVLTest, DeleteAndRebalance) {

    PatientAVL avl;

    avl.addPatient(10, "John", "Fever");

    avl.addPatient(20, "Mark", "Injury");

    avl.addPatient(30, "Sara", "Allergy");


    EXPECT_TRUE(avl.removePatient(10));

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_EQ(avl.getTotalPatients(), 2);

}


// 5. Delete Non-Existing Patient

TEST(PatientAVLTest, DeleteNonExistingPatient) {

    PatientAVL avl;

    avl.addPatient(1, "A", "Cold");

    EXPECT_FALSE(avl.removePatient(50)); // Not in tree

    EXPECT_TRUE(avl.isBalanced());

}


// 6. Get Highest and Lowest Priority Patients

TEST(PatientAVLTest, HighestLowestPriority) {

    PatientAVL avl;
```

```

    avl.addPatient(3, "P1", "Flu");

    avl.addPatient(1, "P2", "Heart Attack");

    avl.addPatient(5, "P3", "Cough");

    Patient high = avl.getHighestPriorityPatient();

    Patient low = avl.getLowestPriorityPatient();

    EXPECT_STREQ(high.name, "P2"); // smallest priority

    EXPECT_STREQ(low.name, "P3"); // largest priority
}

// 7. Range Query (getPatientsInRange)

TEST(PatientAVLTest, RangeQuery) {

    PatientAVL avl;

    avl.addPatient(1, "A", "X");

    avl.addPatient(2, "B", "Y");

    avl.addPatient(4, "C", "Z");

    avl.addPatient(6, "D", "W");

    int count = 0;

    Patient* result = avl.getPatientsInRange(2, 5, count);

    EXPECT_EQ(count, 2); // priorities 2,4

    EXPECT_EQ(result[0].priority, 2);

```

```
    EXPECT_EQ(result[1].priority, 4);

    delete[] result;
}

// 8. Find Non-Existing Patient Returns Null
TEST(PatientAVLTest, FindNonExisting) {

    PatientAVL avl;

    avl.addPatient(10, "Alex", "Cold");

    EXPECT_TRUE(avl.findPatient(99) == nullptr);
}

// 9. Delete All Patients
TEST(PatientAVLTest, DeleteAllPatients) {

    PatientAVL avl;

    avl.addPatient(1, "A", "Condition");
    avl.addPatient(2, "B", "Condition");
    avl.addPatient(3, "C", "Condition");

    avl.removePatient(1);
    avl.removePatient(2);
    avl.removePatient(3);

    EXPECT_EQ(avl.getTotalPatients(), 0);
}
```

```

    EXPECT_TRUE(avl.isBalanced());
}

// 10. Empty Tree Highest/Lowest Should Return Dummy
TEST(PatientAVLTest, EmptyTreeEdgeCase) {
    PatientAVL avl;

    Patient high = avl.getHighestPriorityPatient();

    Patient low = avl.getLowestPriorityPatient();

    EXPECT_EQ(high.priority, -1);

    EXPECT_EQ(low.priority, -1);
}

// 11. Sequential Insertions and Deletions Maintain Balance
TEST(PatientAVLTest, LargeSequence) {
    PatientAVL avl;

    for (int i = 1; i <= 20; ++i)
        avl.addPatient(i, "Temp", "Cond");

    for (int i = 2; i <= 20; i += 2)
        avl.removePatient(i);

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_EQ(avl.getTotalPatients(), 10);
}

```



```

// 12. Reinsert After Deletion Works Correctly

TEST(PatientAVLTest, ReinsertionAfterDelete) {

    PatientAVL avl;

    avl.addPatient(10, "A", "Injury");

    avl.removePatient(10);

    EXPECT_TRUE(avl.addPatient(10, "A", "Recovered"));

    Patient* p = avl.findPatient(10);

    ASSERT_TRUE(p != nullptr);

    EXPECT_STREQ(p->condition, "Recovered");

}

// 13. Tree Height After Multiple Inserts

TEST(PatientAVLTest, HeightAfterInsertions) {

    PatientAVL avl;

    for (int i = 1; i <= 15; ++i)

        avl.addPatient(i, "X", "Cond");

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_LT(avl.getHeight(), 8);

}

```

Lab Task 2: Course Scheduling

Scenario:

Build an **AVL Tree** based course scheduling system where each course is keyed by its **startTime** (an **int** representing minutes since midnight). The AVL property must be preserved after every insertion and deletion. Beyond correctness of the tree, implement efficient domain-specific queries such as finding overlapping courses and computing free time slots.

The registrar needs a scheduler that stores course slots ordered by their start time and supports fast queries and updates. The system must:

- Keep course records balanced so queries remain $O(\log n)$.
- Allow finding all courses that conflict with (overlap) a given time interval.
- Compute all free time intervals (gaps) between courses during a day.
- Support insertion and deletion of courses and always maintain AVL balance.
- Provide diagnostic functions to verify AVL-property and tree height.

Use an **AVL tree keyed by startTime**. If two courses have the same **startTime**, reject the insertion (start times must be unique for simplicity). Times are integers in minutes in range $[0, 1440]$; courses have **startTime** < **endTime**. The day runs from 0 (00:00) to 1440 (24:00).

Data Model and API

```
// Course record
struct Course {
    int courseID;    // unique course identifier
    int startTime;   // minutes since midnight [0, 1440)
    int endTime;     // minutes since midnight, startTime < endTime
    char title[128]; // fixed-size C-string for simplicity
};

// AVL node
struct Node {
    Course data;
```

```

    Node* left;
    Node* right;
    int height;
};

// Scheduler API (member functions of CourseAVL class)
class CourseAVL {
public:
    CourseAVL();
    ~CourseAVL();

    // Core operations
    bool addCourse(int courseID, int startTime, int endTime, const char*
title);
    bool removeCourseByStart(int startTime);    // remove by key
    Course* findCourseByStart(int startTime);    // returns pointer or
nullptr

    // Queries
    // findOverlaps returns dynamic array (allocated with new[]) and sets
outCount.
    // Caller is responsible for deleting the returned array with
delete[].
    Course* findOverlaps(int qStart, int qEnd, int &outCount);

    // getFreeSlots returns an array of pairs (start, end) as Course-like
objects
    // where courseID = -1 and title empty; outCount set to number of
gaps.
    Course* getFreeSlots(int &outCount);

    // Diagnostics
    int getTotalCourses();
    int getHeight();
    bool isBalanced(); // verifies AVL property across all nodes

    // Utility: pretty print (inorder) - prints to stdout
    void displayInorder();

    // (internal helpers are allowed)

```

```
};
```

Requirements & Constraints

1. AVL correctness

- After every `addCourse` and `removeCourseByStart`, the tree must be height-balanced (balance factor $\in \{-1, 0, 1\}$ for every node).
- Rotations (LL, RR, LR, RL) must update `height` correctly.

2. Key uniqueness

- `startTime` is the key. If `addCourse` receives a `startTime` that already exists, it should reject the insert and return `false`.

3. Validation

- Reject invalid times (e.g., `startTime` $\geq$ `endTime`, or times outside `[0, 1440]`).

4. Overlap definition

- Two intervals `[a, b)` and `[c, d)` overlap if `max(a, c) < min(b, d)`.
- Example: `[9:00, 10:00)` and `[9:30, 10:30)` overlap.

5. Free slot computation

- The day is `[0, 1440)`. Free slots are maximal intervals not covered by any course.
- Adjacent courses (`endTime == next startTime`) produce no gap between them.
- If there are no courses, a single free slot `[0, 1440)` is returned.

6. Memory

- No STL containers. Dynamic arrays can be returned via `new` with size communicated via `outCount`.
 - Ensure no memory leaks (delete nodes in destructor).
7. **Efficiency targets**
- `addCourse`, `removeCourseByStart`, `findCourseByStart`: $O(\log n)$ average/worst (due to AVL).
 - `findOverlaps(qStart, qEnd)`: should be $O(k + \log n)$ where k is number of overlapping courses (use interval pruning).
 - `getFreeSlots`: $O(n)$ (inorder traversal once).

Implementation Notes:

1. Store courses keyed by `startTime`.
2. For `findOverlaps`, perform a modified inorder or recursive search:
 - If a node's `startTime` $\geq$ `qEnd`, only search left subtree.
 - If a node's `endTime` $\leq$ `qStart`, only search right subtree.
 - Otherwise, the node overlaps; record it and search both subtrees.
3. For `getFreeSlots`:
 - Traverse nodes in inorder; track `previousEnd` starting at 0.
 - For each node visited with interval `[s, e)`, if `s > previousEnd` add gap `[previousEnd, s)`, then set `previousEnd = max(previousEnd, e)`.
 - After traversal, if `previousEnd < 1440`, add final gap `[previousEnd, 1440)`.
4. For `isBalanced`, compute heights bottom-up and verify balance factor for each node (return false on any violation).
 - Carefully update `height` in rotations and in insert/delete rebalancing code.
5. For deletion of a node with two children, replace with inorder successor (min of right subtree), then rebalance upwards.

```

TEST(CourseSchedulerTest, InsertAndFind) {
    CourseAVL sched;
    bool ok;

    ok = sched.addCourse(101, 540, 600, "CS101"); // 9:00-10:00
    EXPECT_TRUE(ok);
    ok = sched.addCourse(102, 600, 660, "CS102"); // 10:00-11:00
    EXPECT_TRUE(ok);

    Course* c = sched.findCourseByStart(540);
    ASSERT_TRUE(c != nullptr);
    EXPECT_EQ(c->courseID, 101);
    EXPECT_EQ(c->startTime, 540);
    EXPECT_EQ(c->endTime, 600);

    EXPECT_EQ(sched.getTotalCourses(), 2);
    EXPECT_TRUE(sched.isBalanced());
}

TEST(CourseSchedulerTest, RejectDuplicatesAndInvalid) {
    CourseAVL sched;
    bool ok;

    ok = sched.addCourse(201, 480, 540, "Math"); // valid
    EXPECT_TRUE(ok);
    ok = sched.addCourse(202, 480, 520, "Physics"); // duplicate start
    EXPECT_FALSE(ok); // rejected

    ok = sched.addCourse(203, 700, 700, "ZeroLength"); // invalid
    (start==end)
    EXPECT_FALSE(ok);

    ok = sched.addCourse(204, -10, 50, "NegativeStart"); // invalid
    EXPECT_FALSE(ok);
}

TEST(CourseSchedulerTest, FindOverlaps) {
    CourseAVL sched;
    sched.addCourse(1, 540, 600, "A"); // 9:00-10:00
    sched.addCourse(2, 570, 630, "B"); // 9:30-10:30
    sched.addCourse(3, 660, 720, "C"); // 11:00-12:00
    sched.addCourse(4, 600, 660, "D"); // 10:00-11:00

```

```

    int count = 0;
    Course* overlaps = sched.findOverlaps(580, 615, count); // 9:40-10:15

    // Expected overlapping courses: 1 (9:00-10:00), 2 (9:30-10:30), 4
    (10:00-11:00) -> 3 items
    EXPECT_EQ(count, 3);

    // Check that returned items are indeed overlapping and have proper
    times
    for (int i = 0; i < count; ++i) {
        EXPECT_TRUE(max(overlaps[i].startTime, 580) <
min(overlaps[i].endTime, 615));
    }
    delete[] overlaps;
}

TEST(CourseSchedulerTest, GetFreeSlots) {
    CourseAVL sched;
    // Day: 0-1440
    sched.addCourse(10, 480, 540, "Morning"); // 8:00-9:00
    sched.addCourse(11, 600, 660, "LateMorning"); // 10:00-11:00
    sched.addCourse(12, 720, 780, "Noon"); // 12:00-13:00

    int count = 0;
    Course* gaps = sched.getFreeSlots(count);

    // Gaps expected:
    // [0,480), [540,600), [660,720), [780,1440) --> 4 gaps
    EXPECT_EQ(count, 4);

    // Check each gap is valid (courseID == -1 indicates gap)
    EXPECT_EQ(gaps[0].courseID, -1);
    EXPECT_EQ(gaps[0].startTime, 0);
    EXPECT_EQ(gaps[0].endTime, 480);

    EXPECT_EQ(gaps[3].startTime, 780);
    EXPECT_EQ(gaps[3].endTime, 1440);

    delete[] gaps;
}

```

```

TEST(CourseSchedulerTest, DeletionCausesRotation) {
    CourseAVL sched;
    // Insert in order that will create skew and require rotations when
    deleting
    sched.addCourse(1, 300, 330, "A");
    sched.addCourse(2, 200, 230, "B");
    sched.addCourse(3, 100, 130, "C"); // causes imbalance (left-left)
    EXPECT_TRUE(sched.isBalanced());

    // Remove the left-most to force rebalancing on path
    bool ok = sched.removeCourseByStart(100);
    EXPECT_TRUE(ok);
    EXPECT_TRUE(sched.isBalanced());
    EXPECT_EQ(sched.getTotalCourses(), 2);
}

TEST(CourseSchedulerTest, FullDayAndBackToBack) {
    CourseAVL sched;
    // Full day course
    bool ok = sched.addCourse(50, 0, 1440, "FullDay");
    EXPECT_TRUE(ok);

    int count = 0;
    Course* gaps = sched.getFreeSlots(count);
    // No gaps expected
    EXPECT_EQ(count, 0);
    delete[] gaps;

    // Remove full day
    EXPECT_TRUE(sched.removeCourseByStart(0)); // removes startTime == 0
node

    // Back-to-back courses
    sched.addCourse(60, 480, 540, "A"); // 8:00-9:00
    sched.addCourse(61, 540, 600, "B"); // 9:00-10:00 (back-to-back)
    count = 0;
    gaps = sched.getFreeSlots(count);
    // Expect gaps: [0,480), [600,1440) -> 2 gaps
    EXPECT_EQ(count, 2);
    delete[] gaps;
}

```


}